

CSE: MLOps and LLMOps: Engineering AI Systems

Course Outline (Undergraduate) *Tentative*

Institute of Business Administration Karachi
School of Mathematics and Computer Science

Fall 2025

Course Information

- **Course Code:** TBD
- **Course Title:** MLOps and LLMOps: Engineering AI Systems
- **Session:** Fall 2025
- **Credit Hours:** 3 (2-1)
- **Prerequisites (Required):**
 - CSE142: Object Oriented Programming Techniques
 - CSE247: Data Structures
 - CSE307: Introduction to Artificial Intelligence
- **Prerequisites (Recommended):**
 - CSE473: Introduction to Text Analytics (provides foundational LLM knowledge)
- **Course Instructor:** Sualeh Ali
- **Consultation Hours:** To be announced

Course Aims

This course provides a practical, hands-on introduction to the engineering principles required to build, deploy, and maintain modern AI systems. We will start with the foundational practices of Machine Learning Operations (MLOps) for traditional models and then adapt these principles for the unique challenges of Large Language Models (LLMOps). The goal is to equip students with the skills to take a model from a research notebook to a reliable, automated, and monitored production service. The course is heavily tool-driven and culminates in a milestone-based project.

Course Topics

1. The MLOps Lifecycle

- Why MLOps? The gap between ML models and production applications.
- Contrasting the Software Development Lifecycle (SDLC) with the ML Lifecycle.
- Key stages: Data Management, Experiment Tracking, Model Versioning, Deployment, and Monitoring.
- Introduction to the course project: Building a production-ready AI application.

2. Foundational Tools: Git, Environments, and Code Structure

- Version Control with Git: Branching, merging, and collaborating.
- Managing dependencies with Python virtual environments (venv/conda).
- Structuring a Python project for testability and maintainability.
- Example: Setting up a Git repository and project structure for the course project.

3. Containerization and APIs for ML

- Introduction to Docker: Why we need containers for reproducibility.
- Writing a Dockerfile to package an application and its dependencies.
- Building and running Docker images.
- Exposing an ML model as a REST API using FastAPI.
- Example: Containerizing a simple scikit-learn model and serving it with FastAPI.

4. Automation: Continuous Integration (CI)

- Principles of CI/CD: Automating the build and test process.
- Introduction to GitHub Actions for CI.
- Creating a workflow to automatically run tests and build a Docker image on every push.
- Example: Building a CI pipeline for the containerized ML application.

5. Monitoring Traditional ML Models

- Why models fail in production: Data drift and concept drift.
- Setting up monitoring for model performance (e.g., accuracy, F1-score).
- Tools for detecting drift (e.g., Evidently AI).
- Example: Creating a dashboard to monitor a deployed model for data drift.

6. The Shift to LLMOps: What's Different?

- New challenges: Non-determinism, prompt engineering, and the evaluation crisis.

- The rise of Retrieval-Augmented Generation (RAG) to combat hallucinations.
- The role of LLM application frameworks (e.g., LangChain, LlamaIndex).

7. Building LLM Applications with RAG

- The RAG pipeline: Data ingestion, chunking, embedding, and retrieval.
- Introduction to Vector Databases (e.g., ChromaDB, FAISS).
- Building a simple question-answering application over a custom dataset.
- Example: Implementing a basic RAG application using LangChain.

8. Evaluating LLM Applications

- Why traditional metrics (accuracy, etc.) fail for generative tasks.
- The "RAG Triad": Measuring Context Relevance, Groundedness, and Answer Relevance.
- Using LLM-based evaluation ("AI as Judge").
- Hands-on with evaluation frameworks (e.g., TruLens, RAGAs).
- Example: Creating a test set and an automated evaluation suite for the RAG application.

9. CI/CD for LLMs: The Quality Gate

- Integrating the LLM evaluation suite into the CI pipeline.
- Creating a "quality gate" to automatically block changes that degrade performance.
- Versioning prompts alongside code.
- Example: Enhancing the GitHub Actions workflow to run the RAG evaluation suite.

10. Monitoring LLMs: Cost, Latency, and Quality

- Introduction to LLM Observability: Tracing requests through a complex chain.
- Monitoring key metrics: Cost-per-query, token usage, and latency.
- Using observability tools (e.g., LangSmith, Arize Phoenix) to debug and analyze production behavior.
- Example: Instrumenting the RAG application and analyzing traces in an observability tool.

11. Responsible AI and Security

- Understanding failure modes: Bias, toxicity, and prompt injection attacks.
- Implementing basic guardrails and content moderation.
- Data privacy and governance in the age of LLMs.

Milestones

(a) **Milestone 0: Project Pitch**

Students propose their project idea, define the problem they aim to solve, and outline the data and tools they plan to use. Deliverables include a short written proposal (1–2 pages) and a 5-minute pitch presentation.

(b) **Milestone 1: ML Tool Deployment with Monitoring**

Students must develop and deploy a traditional ML model (e.g., classification, regression). The model should be containerized and exposed via an API (FastAPI). They must also implement observability and monitoring tools (e.g., logging, basic dashboards, data drift detection). Deliverables include working code in a repository, a deployed service (local or cloud), and documentation.

(c) **Milestone 2: LLM Application Deployment**

Students extend their system to include an LLM-based component (e.g., RAG application). The system should integrate a vector database for retrieval, demonstrate prompt engineering techniques, and include automated evaluation (using frameworks such as TruLens or RAGAs). Deliverables include a functioning prototype, evaluation results, and documentation of design choices.

(d) **Final Presentation and Report**

At the end of the semester, students will present their complete AI system as a product (ML + LLM components). It should be end to end and preferably with a UI/web interface. The presentation should cover the system architecture, deployment pipeline, monitoring setup, evaluation strategy, and ethical considerations. A final written report (6–8 pages) must accompany the presentation, providing technical details, challenges faced, and lessons learned.

Grading

- **Class Participation** : 10%
- **Quizzes** : 10%
- **Midterm** : 20%
- **Final Milestone-Based Project**: 60%
 - **Milestone 0**: 10%
 - **Milestone 1**: 15%
 - **Milestone 2**: 15%
 - **Final Deliverable and Presentation**: 20%